HW 8 Part 2: Point Clouds, NeRFs, and Gaussian Splats

Daniel Oni | CS 7382 Computer Graphics | Spring 2026

The Three Representations

A **point cloud** is a set of 3D points with positions and sometimes colors or normals. Photogrammetry pipelines like RealityCapture or Polycam start there. Multiple overlapping photos go through Structure-from-Motion, which estimates the camera positions and recovers a sparse point cloud of the scene. From there a meshing step (often Poisson reconstruction) turns the points into triangles, and the triangles get textured from the source photos. The end result is the kind of asset that drops directly into a normal rendering pipeline, which is why photogrammetry has been the default for game engines and scan-to-asset workflows for years.

A **Neural Radiance Field** (NeRF) throws away the explicit geometry. Instead of storing points or triangles, a NeRF trains a small neural network to map any 3D position and viewing direction to a color and a density. To render a pixel, the renderer marches a ray through the volume and samples the network many times along that ray, then accumulates the colors weighted by density. The geometry is implicit; it lives inside the weights of the network. NeRFs handle hard cases like view-dependent lighting and missing camera angles better than photogrammetry, because the network can interpolate. The cost is that training takes time and rendering takes a lot of compute per pixel.

3D Gaussian Splatting takes a different swing at the same problem. The scene is represented as millions of 3D Gaussians, each with a position, a covariance matrix that defines its shape and orientation, an opacity, and a color often expressed using spherical harmonics so it can be view-dependent. Training is gradient-based, similar in spirit to NeRF but operating on these explicit primitives rather than a network. The big difference shows up at render time. Instead of marching rays through a volume, Gaussian Splatting projects each 3D Gaussian onto the image plane (the splatting step), sorts the resulting 2D ellipses by depth, and rasterizes them in tiles. That is a forward mapping, much closer to what we have been doing in WebGL, and it is fast enough to hit real-time framerates on consumer GPUs.

How Gaussian Splats Differ from Photogrammetry

Photogrammetry produces an explicit triangle mesh with a texture atlas. Gaussian Splatting also starts from a sparse point cloud (Structure-from-Motion gives the initial positions), but then it densifies and optimizes Gaussian primitives that are projected and blended rather than meshed. There is no triangle mesh at the end. There are no UV coordinates or texture atlases. The visual fidelity, especially on hair, foliage, and other thin or fuzzy structures, is noticeably better than photogrammetry, because Gaussians can handle soft, semi-transparent geometry that meshes struggle with. The downside is that the result is not directly compatible with traditional pipelines. You cannot just drop a splat into Unity and have it shade the way a mesh would, although plug-ins are catching up.

How These Representations Relate to Rendering

This was the part that took me longest to wrap my head around. Photogrammetry produces a representation that fits inside the classic pipeline we have been studying: vertices, triangles, texture sampling in the fragment shader, all of it. NeRFs and Gaussian Splats both came out of trying to do novel view synthesis without ever building a clean mesh. NeRF is a backward, ray-marched approach where the renderer asks what color a ray should return. Gaussian Splatting is a forward, rasterization-based approach where the renderer projects every primitive into the image plane and blends them in tile order. Both work without a triangle mesh, which answers the assignment question directly: no, geometric representations are not required. What you do need is some scene representation that the renderer can sample or project, plus a rendering algorithm that knows how to compose those samples or projections into pixels.

What Stood Out to Me

The thing I find interesting is that Gaussian Splatting feels closer to what we have been doing in class than NeRF does, even though NeRF came first and gets more attention. Splatting is rasterization, just with soft, view-dependent ellipses instead of opaque triangles. NVIDIA's recent 3D Gaussian Unscented Transform work (3DGUT) even adds support for nonlinear projections like fisheye lenses and hybrid splatting plus ray-tracing for reflections, which starts to merge the two camps. After taking this class, the move from a triangle that gets projected and rasterized to a Gaussian that gets projected and rasterized does not feel like a leap. It feels like the same idea with a fuzzier primitive.

References

- NYT R&D Lab, Gaussian Splatting guide. https://rd.nytimes.com/projects/gaussian-splatting-guide/
- Kerbl et al., 3D Gaussian Splatting for Real-Time Radiance Field Rendering, SIGGRAPH 2023. Reference implementation: https://github.com/graphdeco-inria/gaussian-splatting
- Wu et al., 3DGUT: Enabling Distorted Cameras and Secondary Rays in Gaussian Splatting, CVPR 2025. https://research.nvidia.com/labs/toronto-ai/3DGUT/
- Chen and Wang, A Survey on 3D Gaussian Splatting, arXiv:2401.03890. https://arxiv.org/abs/2401.03890
- Wikipedia, Gaussian splatting. https://en.wikipedia.org/wiki/Gaussian_splatting